HAII 1 - Human AI Interaction 1
Project: Game Assistant

Group Y

Lukas André Feldgrill 11902875
Muhammad Saim Akram 12344542
Jan Zeller 12215076

February 2026

Abstract

This project presents the design and implementation of an LLM-based game assistant for *Minecraft*, aimed at providing in-game assistance through natural language interaction. The system integrates a Retrieval-Augmented Generation (RAG) pipeline to ground model responses in game-specific knowledge, combining a vector database with a large language model to improve factual accuracy and contextual relevance. The RAG database is structured into separate collections for crafting recipes and gameplay guides, enabling more precise retrieval based on user intent.

In addition to the LLM and RAG components, the assistant features speech-to-text and text-to-speech modules, allowing for hands-free interaction, as well as a graphical user interface for input configuration and response visualization. The overall architecture is designed to operate in near real time and performs well on systems equipped with a mid-range GPU.

The assistant currently supports questions related to food, farming, and health mechanics in *Minecraft*. While subjective evaluation shows that the system provides helpful and contextually appropriate responses in most cases, it may occasionally return incomplete or slightly inaccurate answers due to retrieval errors or limitations in the underlying language model. Despite these constraints, the results demonstrate that LLM-based, RAG-enhanced game assistants are a promising approach for improving player support without relying on rigid, scripted systems.

1 Introduction

Video games can be complicated and challenging to pick up. Many players often times lack the time or patience to learn mechanics, read elaborate guides, or watch lengthy tutorials. For those who want to jump in and get immediate, in-game support, we developed a *Game Assistant*.

Large Language Models (LLMs) make it possible to build interactive assistants that can answer questions in natural language. When combined with Retrieval-Augmented Generation (RAG), the model can use a structured knowledge base to produce more accurate and relevant answers. This approach reduces incorrect or fabricated information and makes the responses more grounded in real game data.

In this project, we developed an LLM-based game assistant for *Minecraft*. The assistant focuses on questions related to food, farming, and health mechanics. The system consists of several components: a large language model, a RAG database, a speech-to-text module and a text-to-speech module. The RAG database is divided into two collections (recipes and guides) to better match different types of user questions. Additional mechanisms such as query optimization and intent classification were implemented to improve retrieval accuracy.

2 Project Architecture Overview

The *Game Assitant* was realized in `Python v3.11` and consists of four essential components:

- **Speech-to-Text (STT)**
  Listens and transcribes the users verbal question to text for processing.
- **Retrieval Augmanted Generation (RAG) Database**
  Vector database to store the game information and query relevant context for the user request.

- **Large Languange Model (LLM)**
  The core model that processes both the user question and the RAG provided context to formulate an answer.

- **Text-to-Speech (TTS)**
  Transforms the assitants answer to audio for the user to hear.

Figure 1 shows how these components interact when the user asks a game-related question. The question is first transcribed, detecting when the user stops speaking, and then sent to the RAG database to retrieve context needed to answer it. The user request and retrieved context are combined into a system prompt for the LLM, which generates an answer based on this context. Finally, the response stream is sent to the TTS system and played to the user.

The following sections describe each component in more detail.

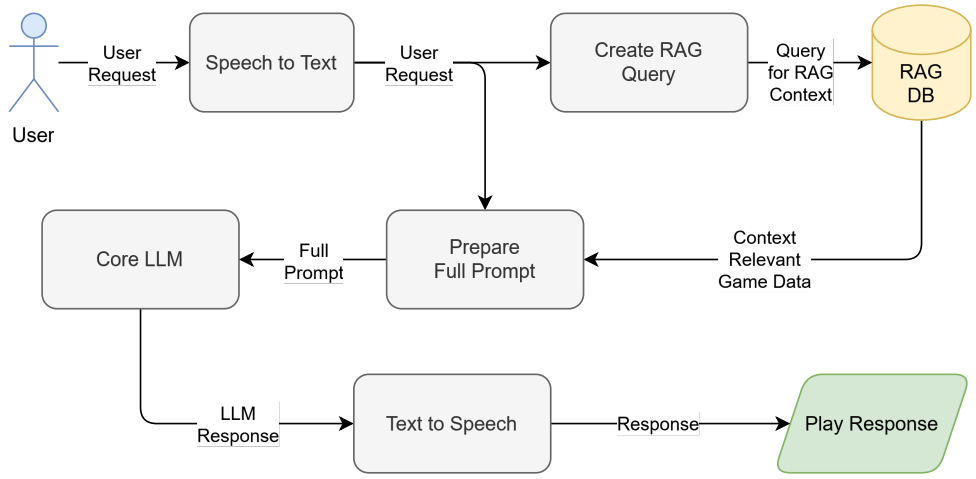


Figure 1: System architecture flow chart of the *Game Assitant*: From the user request to the response.

3 Text-To-Speech

The Text-to-Speech (TTS) module converts the generated LLM responses into spoken audio to complete the Speech-to-Speech pipeline.

3.1 Implementation Logic

We utilized the *pyttsx3*[6] library, a cross-platform TTS engine that operates entirely offline.

- **Configuration**: The engine is initialized with a speech rate of 170 words per minute, selected through testing to provide a clear and natural-sounding pace for game-related information.

- **Feedback Prevention**: A critical implementation detail is the use of blocking calls. This ensures that the TTS finishes speaking before the assistant returns to its listening state, effectively preventing the microphone from picking up the AI's own voice and creating a feedback loop.

3.2 Evaluation

The current implementation uses the default system voice in English. Evaluation shows that the output is **easily understandable**, fulfilling the goal of providing on-demand support that does not disrupt gameplay. Future iterations could focus on more natural-sounding synthetic voices to increase immersion.

4 Speech-To-Text

For the Speech-to-Text (STT) component, we implemented the OpenAI Whisper[5] model, specifically the "base" model. This choice provides a balance between transcription accuracy and computational efficiency for local processing.

4.1 Implementation Logic

We implemented a custom Voice Activity Detection (VAD) system based on audio energy monitoring.

- **Energy-based VAD**: The system continuously monitors microphone input in 100ms frames and calculates the Root Mean Square (RMS) energy.
- **Dynamic Recording**: Recording is triggered when the energy exceeds a defined threshold (0.005) and automatically terminates after 900ms of consecutive silence.
- **Robustness**: This pure-Python approach ensures compatibility across Windows systems.

4.2 Evaluation

Energy-based VAD significantly improved the user experience. The system now facilitates hands-free operation, allowing the user to focus on gameplay while the assistant waits for speech naturally.

5 Core Large Language Model (LLM)

To integrate an LLM into our program, we used the ollama python library [3]. Many of the most popular HuggingFace LLMs are readily available through this library and the connected program. We decided to go with the **qwen3:4b-instruct** model, an instruction tuned model with 4 billion parameters. This ranks as one of the more popular models among the smaller ones.

5.1 Integration and Tasks

The entire pipeline from STT to TTS is handled in our main run_assistant loop. In the beginning, we tell the LLM through a system prompt about its role as a Minecraft assistant and that it should focus on providing short answers for recipes and crafting in general. Whenever user input is picked up, this is fed to the RAG logic and then in combination with the RAG output, the user prompt is provided to the LLM. The LLM output is then given to the TTS in chunks (after every completed sentence). The entire conversation is additionally also stored in a log file.

5.2 Evaluation

Our team is familiar with Minecraft, so the evaluation of the LLM output was done in a subjective manner by manually checking whether the output makes sense and is valid or not. This way, we could see that our final results are accurate most of the time when the questions stay within the scope of food and farming in Minecraft.

6 Retrieval-Augmented Generation (RAG)

Retrieval-Augmented Generation (RAG) combines traditional information retrieval methods (e.g., search engines and databases) with generative Large Language Models (LLMs). By grounding generation in both external data and the model's world knowledge, RAG systems can produce responses that are more accurate, up-to-date, and tailored to the task at hand[8].

Our assistant employs a persistent RAG pipeline built on ChromaDB[1] as the vector database, using the `ONNXMiniLM_L6_V2` embedding model[4] for text representation.

6.1 Data and Structure

The RAG database is divided into two distinct collections: *recipes* and *guides*. Recipes provide structured descriptions of how to craft items from their components, whereas guides describe how to obtain, use, and interact with these items or components, in addition to providing broader gameplay-related knowledge.

This separation allows the system to better tailor retrieved context to different user intents, such as crafting-related questions versus more general gameplay inquiries.

6.2 Recipes

The recipe collection was initially populated with **all** crafting recipes extracted directly from the game files in their original JSON format. Our initial assumption was that the LLM would be able to interpret the JSON structure and generate accurate answers directly from it.

In practice, this approach quickly revealed limitations. The raw JSON format proved poorly suited for embedding with the selected embedding model, leading to significant retrieval issues. These problems were exacerbated by the large number of recipes and by duplication across different crafting stations (e.g., furnace, smoker, and campfire variants for cooking beef).

To address these issues, we deduplicated the dataset by retaining only base recipes and removed all non-food-related recipes. This significantly improved both retrieval accuracy and system performance.

As a final preprocessing step, the remaining recipes were transformed into a human-readable and semantically meaningful text format. Figure2 shows an example of this transformation. The left side displays the processed text file, consisting of a deterministically generated recipe (highlighted in green) and a short natural-language description (highlighted in orange) produced using the same core model (*Qwen3:4b-instruct*) as the assistant. The right side shows the original raw JSON representation (highlighted in red) from the game files.

This processed representation resulted in substantially better embeddings and provided improved contextual information for downstream question answering.

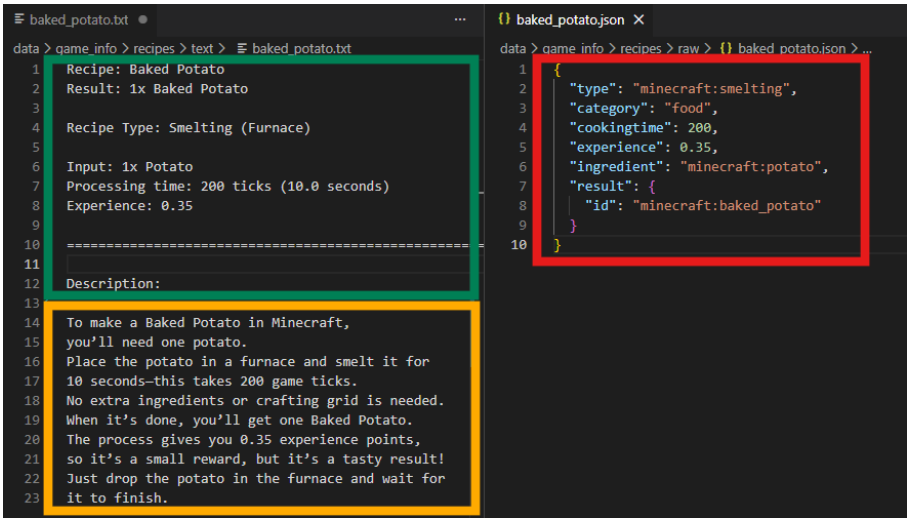


Figure 2: Preprocessing example of the Minecraft recipe for the food item *baked potato*.

6.3 Guides

Relying exclusively on recipes as contextual information proved insufficient, particularly for more technical or exploratory questions such as *"Where can I get X?"* or *"What can I do with X?"*. To address this limitation, we expanded the RAG database by introducing a collection of *guides*.

This addition improved the assistant's performance on non-recipe-related queries. We collected basic guides covering food, farming, and health mechanics in Minecraft from various online sources. Initially, these guides were chunked using a fixed word count with overlap with respect to paragraph endings. Although this approach improved overall accuracy, retrieval errors still occurred, with irrelevant sections occasionally being selected. To avoid this all guides were manually converted into Markdown format, closely preserving the structure of the original sources. The documents were then chunked by subsection rather than by word count, ensuring that each chunk represented a coherent semantic part. Figure3 illustrates this process. Each green box represents a single embedded chunk stored in the database. The guide title and source information are appended to each chunk to maintain traceability and improve contextual grounding.

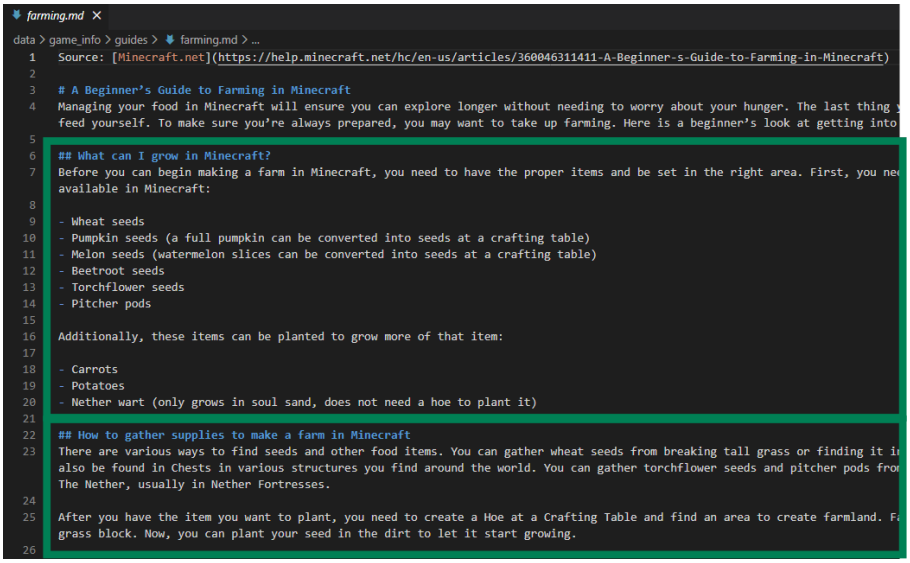


Figure 3: Markdown chunking by subsection for a Minecraft guide on *farming*.

6.4 Data Retrieval

In the initial implementation, user queries were forwarded directly to the RAG database without any pre-processing. This approach proved error-prone, prompting the introduction of two additional mechanisms to improve retrieval accuracy: query optimization and intent classification.

6.4.1 Query Optimization

Using raw user queries for retrieval can be suboptimal, particularly when queries are either very short or overly long. Inspired by prior work and best practices[2, 9], we implemented a *query optimizer* using our core LLM (*Qwen3:4b-instruct*). The optimizer extracts key terms and expands the query to include relevant synonyms and related concepts. For example, the user query:

```
What can I do with an apple?
```

is transformed into

```
apple, uses, functions, crafting, (...), food, apple fruit
```

Although this component is still relatively simple, it resulted in a subjective improvement in retrieval quality.

6.4.2 Intent Classifier

A further challenge arose from maintaining two separate collections (recipes and guides). Initially, the system retrieved a fixed number of results from each collection for every query, regardless of user intent. This often confused the LLM, for instance when a recipe-related question was accompanied by unrelated guide content.

To reduce ambiguity, we implemented an *intent classifier*, again using the core LLM (*Qwen3:4b-instruct*). The classifier determines whether the user is implicitly requesting a recipe, a guide, or both. Based on this classification, the system selectively queries the appropriate collection(s).

Testing on a small evaluation set demonstrated improved relevance and coherence of responses. Future work could further enhance this component by incorporating keyword-based relevance checks on retrieved documents prior to their inclusion in the final context.

6.5 Evaluation

Evaluation of the RAG system was conducted manually. For each newly introduced component or feature, we constructed a set of representative queries and inspected whether the retrieved context contained sufficient and relevant information to answer the question.

This evaluation process was repeated iteratively throughout development to ensure that each modification led to an improvement. Currently, no standardized quantitative metric is used to evaluate context retrieval accuracy; defining such a metric remains an open challenge for future work.

7 Interface

The human-ai interface is a verbal one, the user asks questions and the assistant answers. Additionally, to enhance user experience, we have added a graphical user interface (GUI) which allows users to mute their microphone, select input and output devices, see the status of the assistant and revisit the communication log. See figure 4 for the present prototype. Unfortunately the current implementation of TTS does not allow for the output device to be changed, so the I/O selection is limited to the system standard devices. Another issue is that the mute button is implemented in the main loop, so pressing it may not affect the application instantly, which can cause the assistant to listen to input nonetheless.

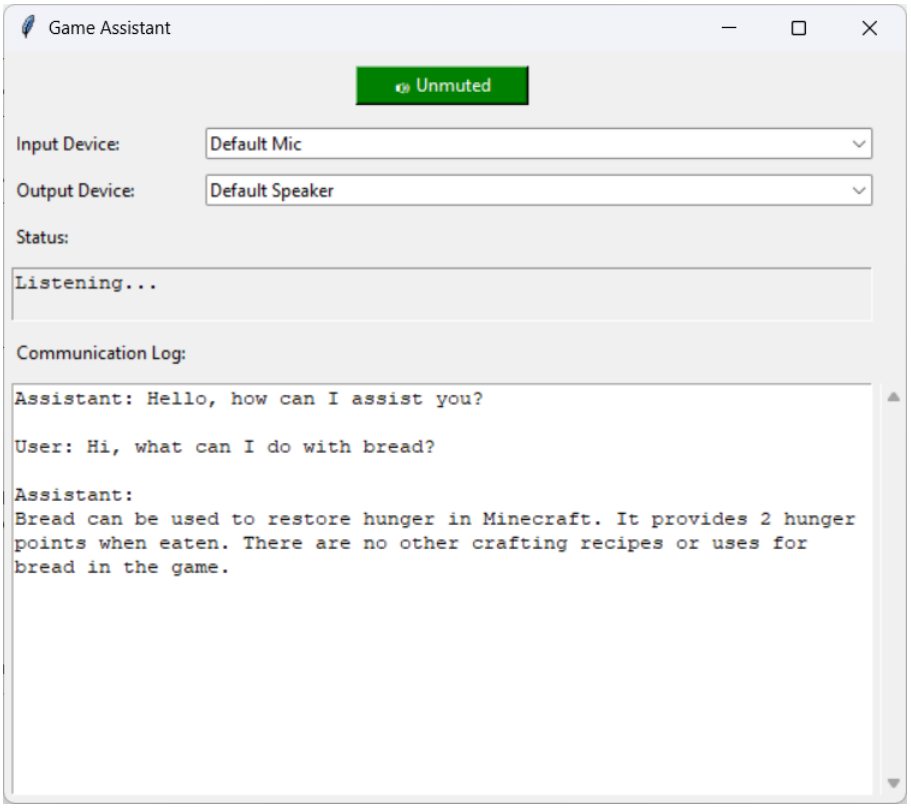


Figure 4: GUI of the Game Assistant.

8 Finished Prototype

The finished prototype can be started through the command line (once all the requirements are installed either manually or using Anaconda). It then shows the GUI mentioned in Section 7. All GUI features except I/O device selection work. Once the system is started up, it will notify the user and they may start speaking. The spoken word is generally captured and translated in textual form very well. The assistant then usually provides accurate information when asked about crafting or cooking in Minecraft. For any other questions, it will mention that it is only a Minecraft assistant, but might still give hallucinated answers to the prompt given. Furthermore, when the user prompt contains particularly friendly phrases, the output might still contain emoji, despite the LLM being instructed in the system prompt to not do so. Though when the user prompt is within the scope of this project, the spoken answers are generally easily understandable and mostly correct. One drawback of the assistant is that it currently does not support interrupting the AI while it is speaking. This results in the need to wait until everything was said. The assistant prototype does not connect to the game and can thus currently not use the context

from the game. For the scope of cooking and crafting, we deemed the game connection to have little merit at this point.

The system runs locally and performs well on computers equipped with a mid-range GPU. While the assistant generally provides helpful answers, it is limited to a specific topic range and may sometimes return incomplete or slightly incorrect information. Nevertheless, this project demonstrates that combining LLMs with retrieval and speech components can create a practical and useful in-game assistant.

9 System Evaluation

We decided to go with a subjective review of the system output to evaluate the results. This is possible because everyone on our development team is familiar with the game. That's why we know which outputs the AI agent should produce and if its responses are correct. This lead us to the result that the answers were mostly correct for our final prototype.

10 Future Works

This section describes the tasks that could be implemented in future works to improve user experience, retrieval accuracy or system performance.

10.1 Expand Scope

The present version only supports a small scope of the game Minecraft: cooking, food and farming. The immediate task would be to scale this, to support all mechanics and aspects of the game. Collect more resources to fill the database with the required information.

10.2 Graph RAG

The current RAG implementation must be updated to Graph RAG, where a complex tree of entities, mechanics and relationships can be projected more efficiently. This improvement is crucial when expanding the scope.

10.3 Orchestrator

Currently, there is hardly any large scale orchestration that helps to precisely classify the intent of a user's request. The question is passed to the RAG to retrieve any matching context, and the RAG tries to classify if the question can be answered by a guide or needs an explicit recipe. The RAG will return at least one result in any case, which might not match the user's question at all, leading to the LLM struggling to formulate a proper response.

For better handling and stream-lined processing, it is advised to have an orchestrator [7] that handles all tasks, i.e. evaluating the user's question, extracting the intent and creating the tasks for each module, or evaluating the RAG results. This approach is already implemented in many large-scale AI applications to improve the output and consequently the user experience.

10.4 Recursive RAG Queries

Sometimes a user question cannot be answered by a single RAG result and may need additional context. The user question: *"What can I do with a rabbit?"* might be answered by the recipe for "Cooked Rabbit", but the assistant would not tell the user that "Cooked Rabbit" can then be used to craft an even better item: "Rabbit Stew".

For cases where an answer might hold back additional information, it would be feasible to do subsequent context searches and give the user the option to ask follow-up questions. This feature could be optionally enabled in the assistant's configuration settings.

10.5 Configuration Options

Every aspect of the assistant is static, so to enhance user experience and adaptability it is required to have some configuration options. Some options might include output voice, tone, level of response details, whether or not to take any game states into account, etc..

10.6 GUI

The GUI does not support switching I/O devices, this should be fixed. Additionally, beatification might be applied, i.e. switching to `customTkinter` as well as adding options to configure the assistant.

10.7 Explainability

Often times explainability is used to help users better understand the answers of an LLM. Adding a form of explainability could improve the user experience when the user also gets the information why the assistant answered the way it did. This feature could be optional to be included in the output and should therefore be configurable.

Resources

[1] *ChromaDB*. URL: `https://www.trychroma.com` (visited on 02/06/2026).

[2] James Luan. *Practical Tips and Tricks for Developers Building RAG Applications*. 2024. URL: `https://zilliz.com/blog/praticial-tips-and-tricks-for-developers-building-rag-applications` (visited on 01/30/2026).

[3] *Ollama's documentation*. URL: `https://docs.ollama.com` (visited on 02/03/2026).

[4] *ONNXMiniLM-L6-V2*. URL: `https://huggingface.co/onnx-models/all-MiniLM-L6-v2-onnx` (visited on 02/06/2026).

[5] *OpenAI Whisper*. URL: `https://github.com/openai/whisper?tab=readme-ov-file` (visited on 02/08/2026).

[6] *pyttsx3*. URL: `https://pyttsx3.com/` (visited on 02/08/2026).

[7] IBM Vanna Winland Joshua Noble. *What is LLM orchestration?* 2025. URL: `https://www.ibm.com/think/topics/llm-orchestration` (visited on 01/30/2026).

[8] *What is Retrieval-Augmented Generation (RAG)?* URL: `https://cloud.google.com/use-cases/retrieval-augmented-generation` (visited on 01/21/2026).

[9] Cheney Zhang. *How to Enhance the Performance of Your RAG Pipeline*. 2024. URL: `https://zilliz.com/learn/how-to-enhance-the-performance-of-your-rag-pipeline` (visited on 01/30/2026).